Title: Comprehensive Non-Uniform Grid Support for MethodOfLines.jl
Organization: NumFOCUS / SciML
Mentors: Chris Rackauckas and Alex Jones

Personal Information & Background

Name: Utku Yılmaz
Email: utkyilmz@gmail.com
SciML Slack: @Utku Yılmaz (Active for GSoC and technical discussions)
Julia Discourse: Utku_Yilmaz
GitHub: utkuyilmaz1903
Project Repository: GSoC-2026-MethodOfLines (Contains all proposal documents, supplementary materials, and PR references)
University & Major: Muğla Sıtkı Koçman University, Computer Engineering (2nd Year)
Time Zone: UTC+3 (Türkiye)
Time Commitment: I will be dedicating 35-40 hours per week to this project. I am fully committed to this timeline and have no overlapping summer internships or heavy academic loads.

Abstract

Finite difference discretization is a cornerstone of numerical PDE solving. While MethodOfLines.jl provides foundational non-uniform grid support via Fornberg weights, its capabilities regarding advanced, high-order schemes remain incomplete. Currently, basic advection schemes lack graceful error handling for ill-posed boundary conditions on non-uniform domains, causing confusing crashes, and advanced high-order schemes, most notably WENOScheme, completely lack non-uniform method dispatch, leading to fatal compilation errors.

The goal of this GSoC 2026 project is to deliver comprehensive, end-to-end non-uniform grid support. This includes refactoring basic schemes to handle variable step sizes, mathematically deriving and implementing native non-uniform stencils for WENO, and conducting extensive performance benchmarks and rigorous convergence tests. While the scope is strictly limited to Finite Difference (FD) formulations, implementing a robust non-uniform grid generator will also lay the essential data structure groundwork for future Finite Volume Method (FVM) integration by accurately tracking varying cell volumes.

Technical Details

Achieving robust non-uniform grid support requires modifications across the core discretization pipeline, specifically interacting with DiscreteSpace and scheme generation logic.

1. Evidence of Feasibility: Breaking the Uniformity Bottleneck

I have developed four strategic prototypes to demonstrate the feasibility of this project and mitigate technical risks:

- **Architectural POC (Draft PR #535):** This PR successfully decoupled construct_differential_discretizer to accept AbstractVector grid types. It proves that the pipeline can ingest irregular coordinate arrays without forcing a scalar *dx*.
- **WENO Engine Prototype (Draft PR #538):** To further validate the numerical core, I implemented a standalone mathematical engine for non-uniform WENO weights featuring dynamic smoothness indicators. It utilizes rigorous Negative Weight Regularization (Shi-Hu-Shu 2002 splitting) to strictly maintain the Partition of Unity ($\Sigma\omega_k = 1.0$) on highly stretched grids. Verified via an MMS suite, it successfully maintains the theoretical O(Δx^3) convergence order. Currently benchmarked at **~83 ns** with 18 allocations, this establishes a mathematically bulletproof baseline that will be optimized to zero-allocation during the integration phase.
- **Boundary Engine Prototype (Draft PR #539):**To provide highly accurate numerical boundary conditions on irregular domains, I developed a zero-allocation, O(1) mathematical engine for 4-point one-sided finite difference weights. Integrating Kahan summation for numerical hardening, it is benchmarked at **~2.0ns** and natively supports Automatic Differentiation (ForwardDiff.Dual) and mixed-type promotion. It has been strictly verified via the Method of Manufactured Solutions (MMS) and maintains bit-perfect mass conservation even under extreme **1:1,000,000,000** grid stretching ratios.
- **Dispatch Infrastructure Prototype (Draft PR #542):** Developed a dispatch hub leveraging Julia's Trait system to route between computational engines. This prototype achieves a performance benchmark of **1.2ns latency and 0 memory allocations**, demonstrating that the architectural transition to non-uniform grids can be handled with absolute zero runtime overhead.

2. The Core Architectural Gap (High-Order Schemes)

Currently, attempting to apply advanced schemes on non-uniform grids results in a hard crash:

```
ERROR: LoadError: Scheme WENO applied to n(t, u)... is not defined.
```

This confirms that the mathematical operators for high-order schemes on variable grids are fundamentally undefined within the DiscreteSpace generator.

3. Proposed Methodology & Architectural Decision

- **Generalizing Weights:** I will bypass the current uniform operator trap by implementing a dynamic branch that calculates local weights node-by-node using the Fornberg algorithm.
- **WENO Mathematics:** Standard WENO relies on smoothness indicators tuned for uniform spacing. Drawing from established literature (e.g., Shu, 1998), I will derive and implement the modified **smoothness indicators (β_k)** and non-linear weights necessary for non-uniform intervals, utilizing Lagrange interpolation based on local geometric distances.
- **Multidimensional Integration:** Following feedback from the core team, the standalone 1D WENO engine is designed to plug directly into the existing multidimensional machinery. Univariate spatial derivatives will act as drop-in stencil providers, ensuring that high-dimensional discretizations inherently support non-uniform configurations without requiring architectural overhauls.
- **Architectural Agreement:** Following a recent technical discussion with core maintainer Chris Rackauckas, we established a clear design boundary. We confirmed that fornberg.jl will remain unmodified as a standard stencil calculator. Instead, this specific non-uniform logic will be implemented as a dedicated, isolated weight calculator entirely within the WENO module, ensuring strict adherence to SciML's modular design philosophy.
- **Multiple Dispatch Routing Strategy:** To ensure **zero breaking changes**, the integration will utilize Julia's multiple dispatch. The system will route to legacy uniform routines for `StepRangeLen` grids, while automatically redirecting `AbstractVector` (Non-uniform) grids to the new isolated weight calculators. This ensures high-performance execution for all users while enabling the new variable-grid capabilities.

Schedule of Deliverables

Community Bonding Period (May 4 - May 24)

- Discuss and finalize the exact mathematical formulations for non-uniform WENOScheme with mentors.
- **Blog Setup:** Publish the introductory post outlining GSoC goals (maintaining a bi-weekly schedule).

Phase 1 (May 25 - July 5)

- **Weeks 1-2:** Audit and refactor core advection schemes to natively support NonUniformGrid indexing (building on PR #533).
- **Weeks 3-4:** Implement the core mathematical dispatch for high-order finite difference weights on variable spacing. *This includes building the isolated non-uniform weight calculator within the WENO module as agreed upon with mentors. I will extend the existing **WENO stress test suite** (developed in PR #538) to cover more complex conservation laws, ensuring the isolated calculator remains robust under extreme clustering before full integration.*
- **Weeks 5-6:** Plug the 1D non-uniform WENO stencils into the existing multidimensional codegen machinery. Implement rigorous convergence tests to verify that the new non-uniform WENO scheme maintains its theoretical order of accuracy across higher-dimensional PDEs.
- **Phase 1 Deliverables:** Stable basic schemes, initial PR for 1D non-uniform WENOScheme, convergence test suite.
- **Blog Deliverables:** 3 technical posts
  Proposed topics:
  1. *Deriving Non-Uniform Smoothness Indicators (β_k)*
  2. *Dynamic Weight Calculation with Fornberg*
  3. *Handling Non-Uniform Boundaries in Multi-Dimensional PDEs*

Phase 2 (July 6 - August 2)

- **Weeks 7-8 (Advanced Integration):** Address Mid-Term feedback, resolve edge cases (e.g., extreme clustered grid ratios), and optimize weight generation.
- **Weeks 9-10 (Benchmarking & Capstone):** Integrate BenchmarkTools.jl to profile execution times against uniform counterparts and validate L_2 error norms.
  Capstone Test Case: I will build a multi-domain PDE test case featuring high boundary gradients (inspired by battery interface dynamics). This will prove the new infrastructure resolves steep gradients accurately, focusing solely on validating the numerical method performance.
- **Phase 2 Deliverables:** Fully optimized non-uniform stencils, passing CI/CD pipelines.
- **Blog Deliverables:** 2 benchmarking posts
  Proposed topics:
  1. *Profiling Dynamic Stencil Allocations in Julia*
  2. *L_2 Error Norm Convergence Analysis*

Final Week (August 3 - August 16)

- Clean up open PRs, merge code, expand official MethodOfLines.jl tutorials for custom non-uniform grids (building on docs PR #534), and submit the final report.

Development Experience & Contributions

To prepare, I have deeply familiarized myself with the SciML codebase, directly tackling architectural bugs.

[Merged] Core Architectural & Dispatch Fixes

- **SciMLBase.jl / PR #1265 / Issue #1130 (Resolved):** Prevented silent CPU fallbacks in GPU simulations by fixing a critical dispatch trap in EnsembleProblem, implementing base-level architectural validation.
- **SciMLBase.jl / PR #1258:** Fixed a MethodError by dynamically typing analytic functions in AutoSpecialize.
- **MethodOfLines.jl / Issue #445 (Resolved):** Solved a critical mass leakage problem in age-of-infection SIR models by reformulating the system algebraically as a DAE to enforce mass conservation.

[Open/Active] Grid & Discretization Refactoring (MethodOfLines.jl)

- **PR #533:** Fixing a critical BoundsError in UpwindScheme by correcting hardcoded D.offside logic for boundary coefficients on non-uniform grids.
- **PR #532:** Preventing infinite loops in transform_pde_system! by fixing boolean context errors with SymbolicUtils.
- **PR #534:** Added a foundational mathematical tutorial for non-uniform grid discretization to the official documentation.
- **PR #541 & PR #543:** Investigated a complex `RuleRewriteError` masking a `BoundsError` in `UpwindScheme` on non-uniform domains. Following direct mentorship feedback, I identified that masking the error was the wrong architectural approach. I immediately architected and submitted **PR #543**, which introduces a recursive well-posedness check early in the MOL discretization flow. This natively catches missing boundary conditions and throws a graceful `ArgumentError` before code generation, proving my ability to adapt to mentor feedback and implement structural pipeline improvements.

[Open/Active] Core Symbolic Engine (SymbolicUtils.jl)

- **PR #884 for Issue #876:** Fixed a scoping leak in nested `ArrayOp` substitution during scalarization. By implementing a scope-aware substitution filter, I prevented outer index values from incorrectly overriding inner bindings. This patch was necessary to unblock the evaluation of non-linear diffusion and advanced stencils (like WENO) in MethodOfLines.jl.

Personal R&D (The learning-sciML-pde Repository)

To systematically prepare for this project, I maintain a public learning repository containing 8 progressive mini-projects focused on numerical PDE discretizations. Key highlights directly supporting this GSoC proposal include:

- **WENO Scheme Prototyping (Mini-Project 6):** Mathematically analyzed and prototyped non-uniform β_k smoothness indicators, proving the failure of classic fractional weights (e.g., 13/12) on clustered grids and verifying the dynamic Lagrange approach.
- **Fornberg Benchmarking (Mini-Project 5):** Built a custom Fornberg implementation, proving via BenchmarkTools.jl that dynamic node-by-node weight calculation introduces zero memory allocation overhead compared to uniform grids.
- **Non-Uniform Advection & FVM Geometry (Mini-Projects 4 & 8):** Implemented advection-diffusion models on clustered vector grids (which directly led to my bug discovery and fix in PR #533) and built staggered grid geometries to verify mass conservation for future FVM capabilities.

Why SciML & Why This Project?

Building the mathematical infrastructure for non-uniform grids is not just a coding task; it is an engineering necessity for simulating real-world physics accurately. While investigating MethodOfLines.jl, I realized that bridging this architectural gap would unlock a massive level of precision for the scientific computing community.

I want to dedicate my summer to this project because SciML represents the bleeding edge of scientific machine learning. Successfully implementing high-order non-uniform schemes requires a rigorous blend of theoretical mathematics and high-performance Julia code. Having already successfully merged critical architectural fixes across the SciML ecosystem, I am fully prepared to transition from fixing isolated bugs to delivering a core discretization pipeline under the guidance of world-class mentors.